IT-Projektportfolio

Juan Miguel Rodriguez Ramirez

Projektübersicht

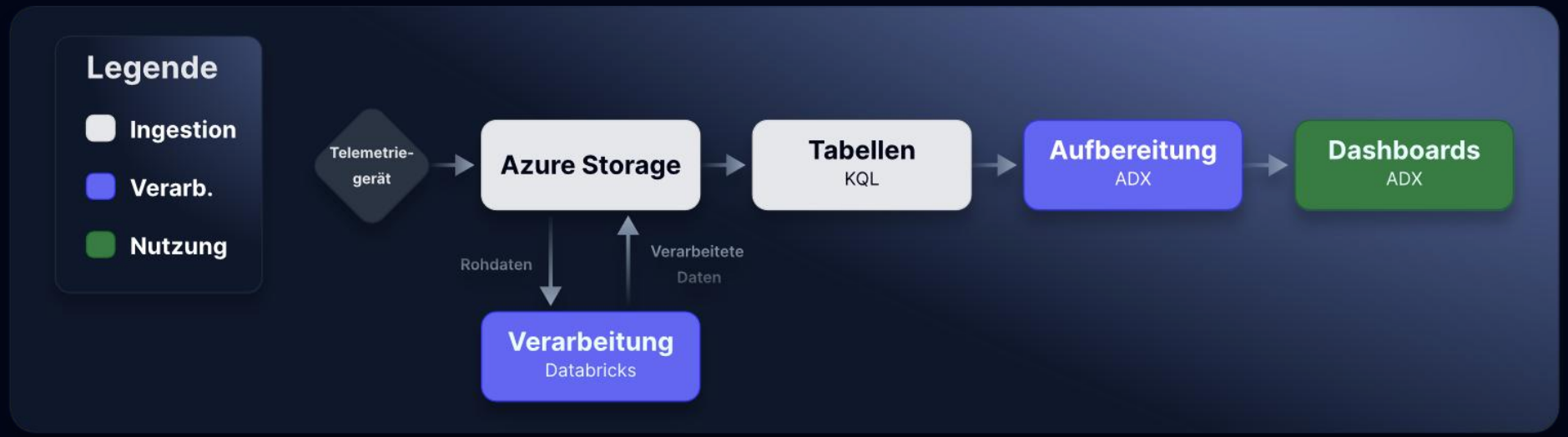
Titel	Schwerpunkt	Technologien
Cloudbasierte Datenpipeline für Fahrzeug-Telemetrie	Data Engineering Pipeline	Azure, ADX, KQL, Python
API zur Benutzer-Provisionierung	Enterprise Backend API	Java, SCIM, Azure AD, PostgreSQL
Full-Stack E-Commerce Plattform	Full-Stack Web App	Java, Spring Boot, Angular, MySQL
Echtzeit-Mehrsprachiges Sprachübersetzungssystem	Full-Stack Web App	Azure, Python, Streamlit
Probabilistische Zuverlässigkeitsmodell für Wasseraufbereitung	Machine Learning Modellierung	MATLAB, GeNIe
Trainingsprotokoll-Verwaltungssystem	Backend Data App	Python, PostgreSQL, NumPy
Spielesammlung & Fortschritts-Tracker	Backend Data App	C++, SQLite

Cloudbasierte Datenpipeline für Fahrzeug-Telemetrie

Beschreibung & Systemarchitektur

Cloudbasierte Datenpipeline für Fahrzeug-Telemetrie

Ablaufdiagramm



Cloudbasierte Datenpipeline für Fahrzeug-Telemetrie

Einleitung

Kontext und Herausforderung

Hochfrequente Telemetriedaten aus verteilten Sensorsystemen erzeugten große Mengen inkonsistenter Zeitreihenlogs. Eine cloud-native Pipeline war erforderlich, um diese Daten effizient zu ingestieren, vorzuverarbeiten und für Analyse und Visualisierung zu strukturieren.

Zielsetzung und Rolle

Entwicklung einer skalierbaren Telemetrie-Pipeline zur Umwandlung roher Gerätelogs in analysefähige Datensätze und Dashboards.
Rolle: Data Engineer verantwortlich für Datenmodellierung, KQL-Vorverarbeitung, ADX-Dashboards und analytische Unterstützung.

Cloudbasierte Datenpipeline für Fahrzeug-Telemetrie

Methodik

Lösungsansatz

Eine cloud-first Pipeline verarbeitete rohe Telemetriedaten in Azure Storage und ADX, mit KQL für Verarbeitung, Verfeinerung und Analyse. Zusätzliche Transformationen erfolgten in Python und Databricks.

Architektur und Implementierung

- Zentralisierung roher Telemetriedaten in Azure Storage und ADX.
- Vorverarbeitung, Feature-Extraktion und Normalisierung direkt in KQL implementiert.
- ADX-Dashboards für nahezu Echtzeit-Visualisierung und Analyse.

Ergebnisse und Wirkung

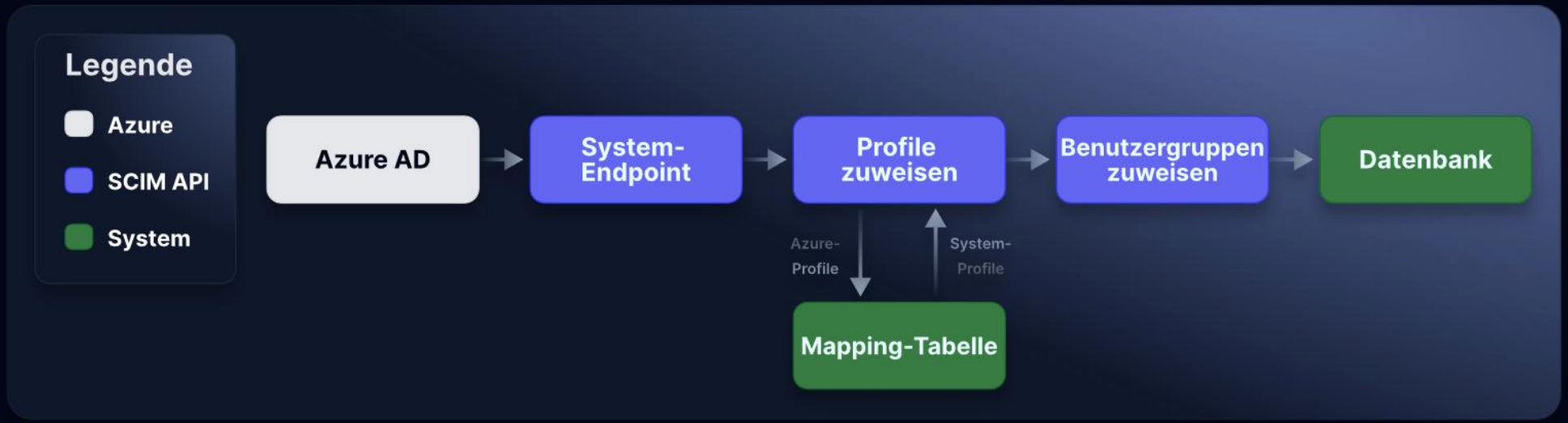
- Nahezu Echtzeit-Analyse des hochfrequenten Sensorverhaltens.
- Reduzierter manueller Vorverarbeitungsaufwand durch automatisierte KQL-Transformationen.
- Verbesserte Fehlersuche und Nachvollziehbarkeit durch strukturierte Datensätze und Dashboards..

API zur Benutzerprovisionierung

Beschreibung & Systemarchitektur

API zur Benutzer-Provisionierung

Ablaufdiagramm



API zur Benutzer-Provisionierung

Einleitung

Kontext und Herausforderung

Unternehmensumgebungen nutzen Azure Active Directory (Azure AD) zur Verwaltung von Benutzeridentitäten und Lebenszyklusereignissen. Eine Backend-Integration war erforderlich, um Benutzer, Teams und Berechtigungen zuverlässig anhand von Provisioning-Ereignissen zu synchronisieren und dabei Organisationsstruktur und Zugriffskonsistenz zu wahren.

Zielsetzung und Rolle

Entwurf und Implementierung eines Backend-Services zur Verarbeitung von Azure-AD-SCIM-Provisioning-Ereignissen, zur Zuordnung von Benutzern zu internen Rollen und Teams sowie zur Synchronisierung der Benutzerzustände.

Rolle: Backend Engineer mit Verantwortung für API-Design, Provisioning-Logik, Datenmapping und Datenbankaktualisierungen.

API zur Benutzer-Provisionierung

Methodik

Lösungsansatz

Eine SCIM-kompatible REST-API wurde mit Java und Spring Boot implementiert, um Azure-AD-Provisioning-Ereignisse zu verarbeiten und Identitäten deterministisch internen Rollen und Teams zuzuordnen.

Architektur und Implementierung

- SCIM-kompatible REST-Endpunkte zur Verarbeitung von Benutzer-Lifecycle-Ereignissen aus Azure AD.
- Mapping-Logik zur Übersetzung von Gruppen und Attributen in interne Rollen, Teams und Hierarchien.
- Persistierung von Benutzerstatus und Berechtigungen in PostgreSQL mit Hibernate/JPA.

Ergebnisse und Wirkung

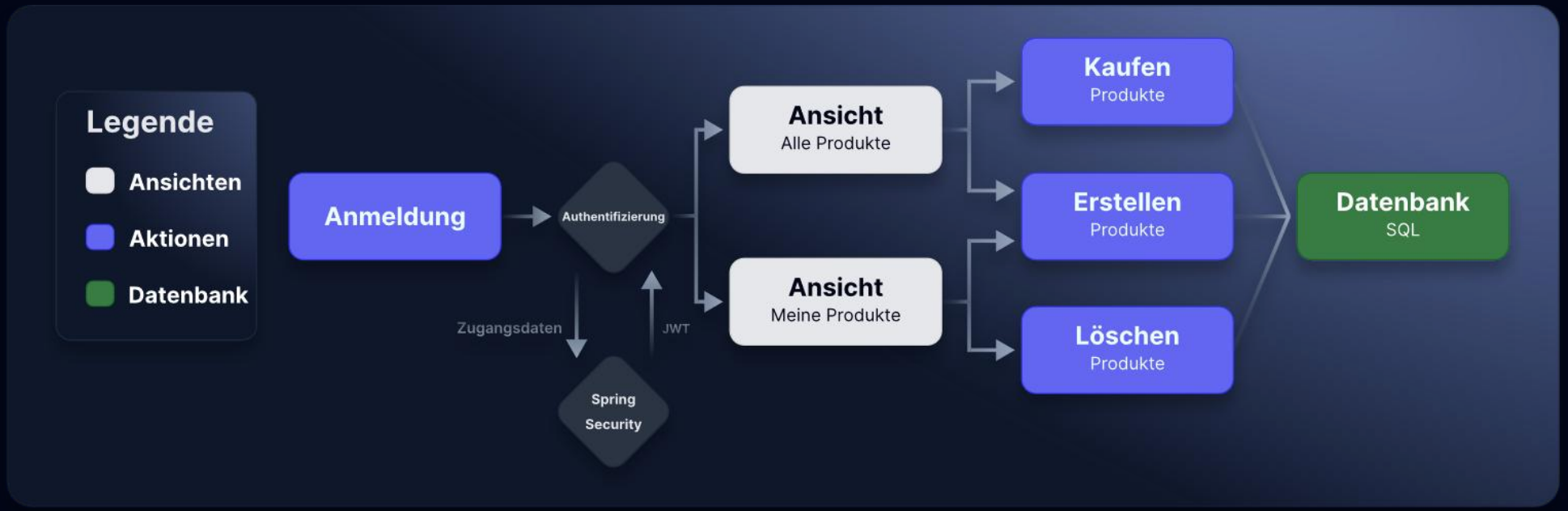
- Zuverlässige Provisioning-Integration zur Synchronisierung von Azure-AD-Identitäten.
- Automatisierung von Benutzern, Updates und Deaktivierung, mit reduziertem Administrationsaufwand.
- Konsistente und auditierbare Rollen- und Berechtigungszuweisung über alle Teams hinweg.

Full-Stack E-Commerce Plattform

Beschreibung & Systemarchitektur

Full-Stack E-Commerce Plattform

Ablaufdiagramm



Full-Stack E-Commerce Plattform

Einleitung

Kontext und Herausforderung

Kleine Händler verfügen häufig nicht über ein einheitliches System zur Verwaltung von Produkten, Lagerbeständen und Verkaufstransaktionen. Eine Full-Stack-Lösung war erforderlich, um Authentifizierung, Bestandsverwaltung, Käufe und die persistente Speicherung von Transaktionen in einer relationalen Datenbank zu unterstützen.

Zielsetzung und Rolle

Entwurf und Implementierung einer Full-Stack-E-Commerce-Anwendung mit einer sicheren Backend-API, einem responsiven Frontend und konsistenter Persistenz von Bestands- und Kaufdaten.
Rolle: Full-Stack Developer verantwortlich für Backend, Frontend, Datenbankintegration und Deployment.

Full-Stack E-Commerce Plattform

Methodik

Lösungsansatz

Ein REST-basiertes Backend in Java mit Authentifizierung wurde umgesetzt und mit einem Angular-Frontend für Produkt- und Kaufprozesse kombiniert. Das System wurde containerisiert.

Architektur und Implementierung

- Relationales Datenmodell für Benutzer, Produkte und Kauftransaktionen.
- REST-APIs mit Spring Boot, Hibernate/JPA und JWT-gesicherten Endpunkten.
- Responsives Angular-Frontend und vollständiges Stack mit Docker containerisiert.

Ergebnisse und Wirkung

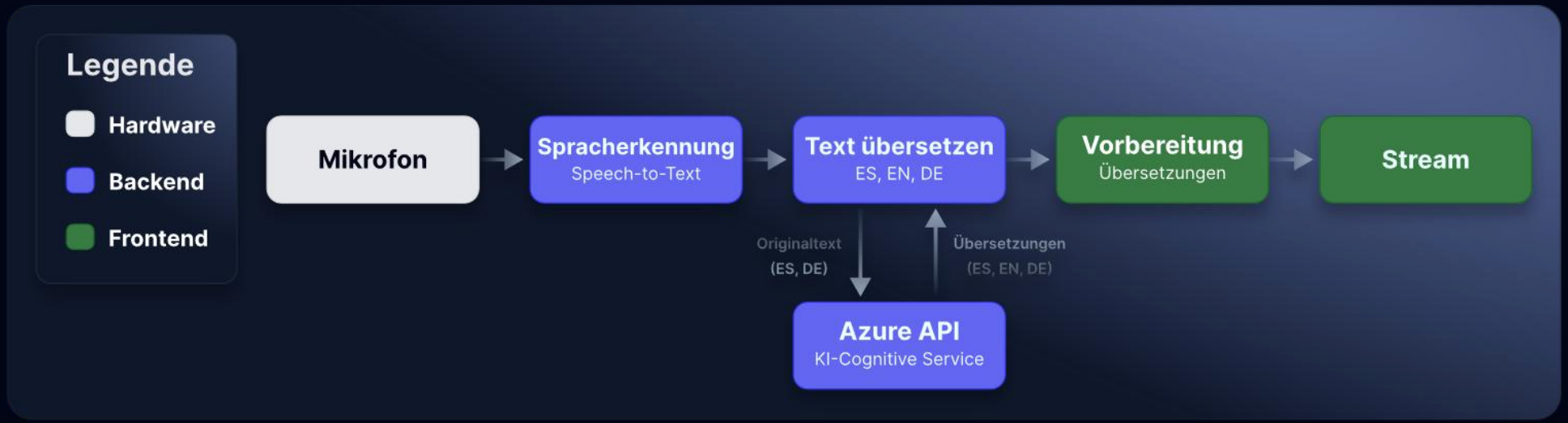
- Funktionsfähige End-to-End-E-Commerce-Anwendung mit persistierten Transaktionen.
- Nachweis von Full-Stack-Kompetenz über Backend, Frontend und Datenbank hinweg.
- Einsatzfähige Architektur für lokale und iterative Entwicklung.

Echtzeit-Mehrsprachiges Sprachübersetzungssystem

Beschreibung & Systemarchitektur

Echtzeit-Mehrsprachiges Sprachübersetzungssystem

Ablaufdiagramm



Echtzeit-Mehrsprachiges Sprachübersetzungssystem

Einleitung

Kontext und Herausforderung

Live-Veranstaltungen sind häufig für Teilnehmende unzugänglich, die die Hauptsprache nicht sprechen. Ein Echtzeitsystem war erforderlich, um gesprochene Sprache zu erfassen, zu transkribieren, in mehrere Sprachen zu übersetzen und während der Veranstaltung gut lesbare Untertitel auf externen Bildschirmen anzuzeigen.

Zielsetzung und Rolle

Entwicklung einer Anwendung für Echtzeit-Spracherkennung, mehrsprachige Übersetzung und browserbasierte Anzeige von Untertiteln.
Rolle: Full-Stack Developer verantwortlich für Audioerfassung, Transkription, Übersetzung und Visualisierung.

Echtzeit-Mehrsprachiges Sprachübersetzungssystem

Methodik

Lösungsansatz

Eine Python-Pipeline wurde entwickelt, um Live-Audio zu erfassen und transkribieren, Text zu übersetzen und Untertitel in Echtzeit darzustellen. Das System unterstützt austauschbare Ein- und Ausgabesprachen.

Architektur und Implementierung

- Kontinuierliche Audioerfassung und Streaming in eine Speech-to-Text (STT) Pipeline mit Azure-Diensten.
- Integration von STT- und Übersetzungsdiensten zur Erzeugung mehrsprachiger Untertitel in Echtzeit.
- Streamlit-basiertes Frontend zur Anzeige von Live-Untertiteln im Browser pro Zielsprache.

Ergebnisse und Wirkung

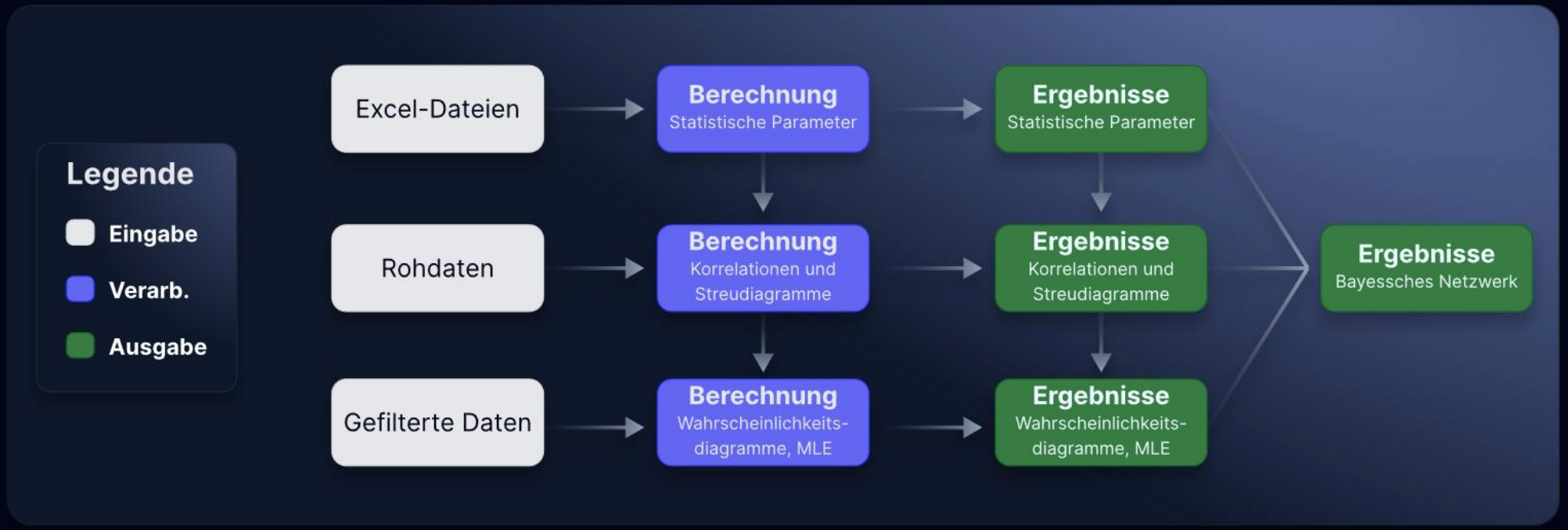
- Wiederverwendbare Anwendung für mehrsprachige Echtzeit-Untertitelung bei Live-Events.
- Ermöglichte das Mitverfolgen gesprochener Inhalte für nicht-muttersprachliche Teilnehmende.
- Integration von Audio-Streaming, STT, maschineller Übersetzung und webbasierter Visualisierung.

Probabilistische Zuverlässigkeitsmodell für Wasseraufbereitung

Beschreibung & Systemarchitektur

Probabilistische Zuverlässigkeitsmodell für Wasseraufbereitung

Ablaufdiagramm



Probabilistische Zuverlässigkeitsmodell für Wasseraufbereitung

Einleitung

Kontext und Herausforderung

Mehrstufige Abwasserbehandlungssysteme müssen unter erheblicher betrieblicher Variabilität und Messunsicherheit strenge Anforderungen an die Schadstoffentfernung erfüllen. Die Bewertung der Systemkonformität erfordert die Propagation von Unsicherheiten über alle Behandlungsstufen hinweg.

Zielsetzung und Rolle

Entwicklung eines probabilistischen Zuverlässigkeitsmodells zur Abschätzung der Wahrscheinlichkeit, dass die gesamte Behandlungskette die Zielwerte der Schadstoffentfernung erfüllt.
Rolle: verantwortlich für Datenaufbereitung, stochastische Modellierung, Design von Bayes'schen Netzen und Zuverlässigkeitsanalyse.

Probabilistische Zuverlässigkeitsmodell für Wasseraufbereitung

Methodik

Lösungsansatz

Ein datengetriebener probabilistischer Ansatz wurde entwickelt, der stochastische Simulation und bayesische Inferenz zur Quantifizierung von Konformitätswahrscheinlichkeiten kombiniert.

Architektur und Implementierung

- Aufbereitung von Felddaten und Anpassung statistischer Verteilungen mit MATLAB.
- Erzeugung korrelierter stochastischer Stichproben mittels Nataf-Transformation.
- Integration von Variablen in ein Bayes'sches Netzwerk in GeNIe zur Abhängigkeitsmodellierung.

Ergebnisse und Wirkung

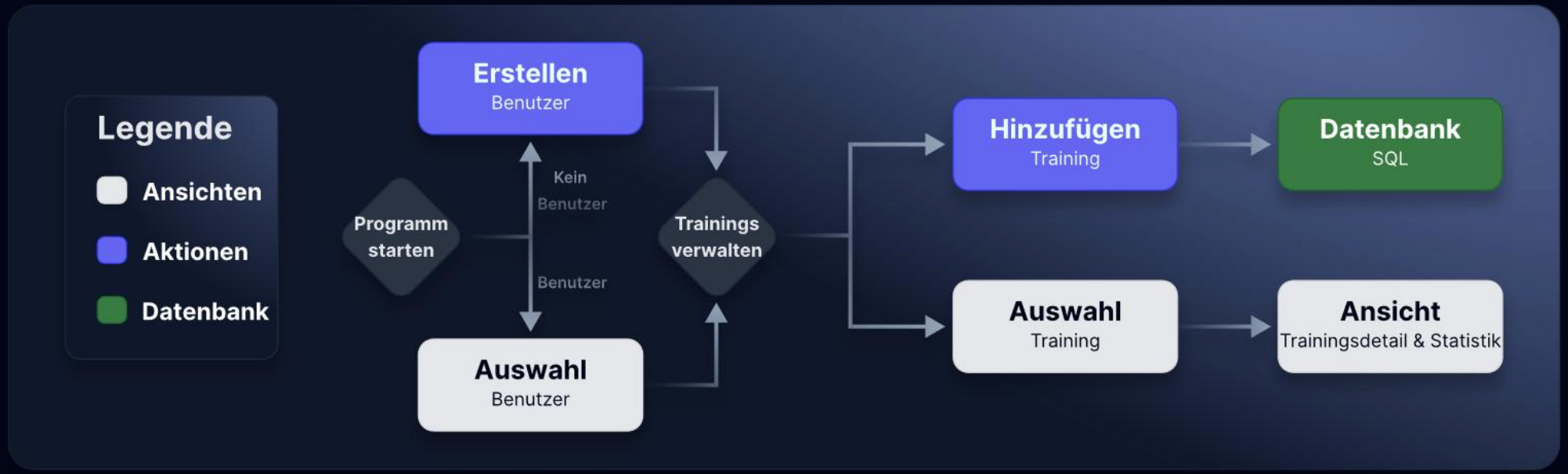
- Vorhersage der Systemleistung und der Einhaltung von Grenzwerten unter Unsicherheit.
- Quantifizierung der Zuverlässigkeit pro Behandlungsstufe und für das Gesamtsystem.
- Identifikation dominanter Unsicherheitsfaktoren zur risikobasierten Optimierung.

Trainingsprotokoll-Verwaltungssystem

Beschreibung & Systemarchitektur

Trainingsprotokoll-Verwaltungssystem

Ablaufdiagramm



Trainingsprotokoll-Verwaltungssystem

Einleitung

Kontext und Herausforderung

Die Erfassung von Trainingseinheiten in Notizbüchern oder Tabellen führt zu unstrukturierten Daten, eingeschränkten Analysefunktionen und geringer Portabilität zwischen Speichersystemen. Eine leichte Lösung war erforderlich, um Trainingsdaten zuverlässig zu speichern und historische sowie volumenbasierte Analysen zu ermöglichen.

Zielsetzung und Rolle

Entwurf und Implementierung eines Trainingsprotokollsystems mit Mehrbenutzerunterstützung, SQL-Datenpersistenz und strukturierter Auswertung von Trainingshistorie und Statistiken.
Rolle: verantwortlich für Datenmodellierung, Anwendungslogik und Datenbankintegration.

Trainingsprotokoll-Verwaltungssystem

Methodik

Lösungsansatz

Eine Python-basierte Konsolenanwendung wurde mit klarer Trennung von Anwendungslogik und Datenzugriff entworfen, sodass dieselbe Kernlogik auf unterschiedlichen SQL-Backends durch Austausch des Datenbankmoduls genutzt werden kann. Eine menügesteuerte Oberfläche führt durch Kontoverwaltung, Protokollierung und Datenauswertung.

Architektur und Implementierung

- Relationales Datenmodell mit Benutzern, Übungen und Trainingslogs, verknüpft über Fremdschlüssel.
- Zentrale Anwendungslogik in Python, mit getrennten SQL-Abfragen und Visualisierungen.

Ergebnisse und Wirkung

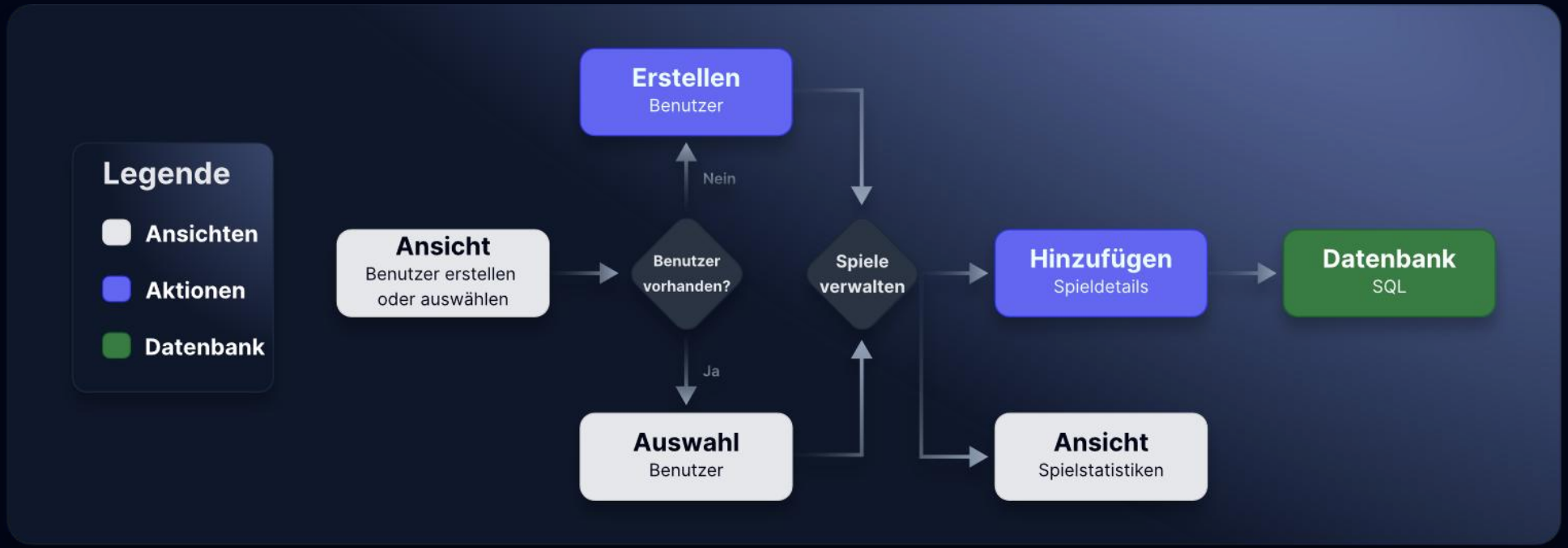
- Funktionsfähiger, portabler Prototyp zur Erfassung und Abfrage strukturierter Trainingsdaten.
- Historische und zeitfensterbasierte Analyse von Trainingsvolumen und -fortschritt.

Spielesammlung & Fortschritts-Tracker

Beschreibung & Systemarchitektur

Spielesammlung & Fortschritts-Tracker

Ablaufdiagramm



Spielesammlung & Fortschritts-Tracker

Einleitung

Kontext und Herausforderung

Die manuelle Verwaltung von Spielesammlungen in Listen führt zu fragmentierten Daten und eingeschränkter Suchbarkeit. Eine strukturierte lokale Anwendung war erforderlich, um Spieldaten zu persistieren, Spielstatus zu verwalten und zuverlässige Abfragen zu ermöglichen.

Zielsetzung und Rolle

Entwicklung einer Desktop-Anwendung zur Verwaltung von Spielmetadaten, Benutzerstatus und Fortschritt mit persistenter Speicherung in SQL.
Rolle: verantwortlich für Datenmodell, Anwendungslogik, Benutzeroberfläche und SQL-Integration.

Spielesammlung & Fortschritts-Tracker

Methodik

Lösungsansatz

Eine lokale Desktop-Anwendung mit klarer Trennung zwischen Anwendungslogik und persistenter Speicherung, die eine strukturierte Verwaltung von Spielesammlungen und Benutzerfortschritt ermöglicht.

Architektur und Implementierung

- Relationales SQLite-Schema für Benutzer, Spiele, Status und Benutzer–Spiel-Beziehungen.
- C++ Datenzugriffsschicht zur Umsetzung von CRUD-Operationen über die SQLite C++ API.
- Anwendungslogik für Statusübergänge, Katalogabfragen und Verwaltung von Wunschlisten.

Ergebnisse und Wirkung

- Funktionsfähige Desktop-Anwendung zur Verwaltung von Spielen und zugehörigen Statusinformationen.
- Skalierbare Grundlage für zukünftige Erweiterungen wie Bewertungen, Tags oder Analysen.

Kontaktdaten

Juan Miguel Rodriguez Ramirez

juanm.rodriguez.dev@gmail.com

www.test.com